

UAH Mars Drone ASW

Architecture Design Document

Yihune García García

May 31, 2026

Contents

1	Introduction	3
2	Applicable and Reference Documents	3
3	Terms, Definitions and Abbreviated Terms	3
4	Software Design Overview	3
5	Software Design	4
5.1	Software Static Architecture	4
5.2	Software Dynamic Architecture	4
5.2.1	Computational Model	4
5.2.2	Software Behaviour	5
5.3	Real-Time Requirements	6
5.4	Software Components Design	6
5.4.1	CCDroneMng	6
5.4.2	Interfaces	7
5.4.3	Behaviour	8
5.5	SRD Req Coverage Matrix	11
6	Traceability	11

1 Introduction

This document describes the architectural design of the *UAH Mars Drone ASW* application software. The document focuses on the `CCDroneMng` component, which is responsible for the drone management behaviour, including drone configuration, telecommand execution, flight plan initialization, periodic control execution and return to ready state.

The content of this document has been obtained from the EDROOM model and from the generated source code of the project. The objective is to complete the Architectural Design Document required for the final project milestone.

2 Applicable and Reference Documents

Reference	Document
AD-01	UAH Mars Drone ASW Project Statement.
AD-02	Final Milestone Project Instructions.
AD-03	ADD Template provided for the project.
AD-04	EDROOM model of the UAH Mars Drone ASW.
AD-05	Generated source code of <code>CCDroneMng</code> .
AD-06	PUS Service 129 implementation.

Table 1: Applicable and reference documents

3 Terms, Definitions and Abbreviated Terms

Term	Meaning
ADD	Architectural Design Document.
ASW	Application Software.
DDF	Design Definition File directory.
DJF	Design Justification File directory.
EDROOM	Event Driven Real-Time Object Oriented Methodology.
PUS	Packet Utilisation Standard.
TC	Telecommand.
TM	Telemetry.
HK	Housekeeping telemetry.
FDIR	Fault Detection, Isolation and Recovery.
ST[129]	Private PUS Service 129, dedicated to drone management.

Table 2: Terms and abbreviations

4 Software Design Overview

The UAH Mars Drone ASW is structured as an EDROOM component-based real-time application. Each component communicates through typed ports and protocols, and its behaviour is defined by a state machine.

The `CCDroneMng` component is the main component responsible for the drone management logic. Its main responsibilities are:

- Performing the initial drone self-test.
- Receiving the initial drone configuration.
- Replying that the drone is ready after configuration.
- Receiving and executing drone telecommands.
- Starting the flight plan when it is ready.
- Executing the periodic flight control algorithm.
- Returning the drone to the ready state when the flight plan is completed.

The periodic flight control loop is driven by the `DroneTimer` timing port. The next control instant is stored in the internal variable `VNextCtrl`.

5 Software Design

5.1 Software Static Architecture

The static architecture is defined by the EDROOM component model. The component documented in this ADD is `CCDroneMng`. This component has an external control port named `DroneMngCtrl` and a timing port named `DroneTimer`.

The `DroneMngCtrl` port uses the protocol `CPDroneMngCtrl`. This protocol defines the input messages `SDroneSetUp` and `SDroneTC`, and the output message `SDroneReady`. The `DroneTimer` port is used to receive the timeout event that triggers the periodic flight control algorithm.

5.2 Software Dynamic Architecture

5.2.1 Computational Model

The computational model of `CCDroneMng` is event-driven. The component remains waiting for EDROOM messages. Depending on the current state and the received signal, the component takes the corresponding transition and executes the associated action.

The main operational states of `CCDroneMng` are:

- `SelfTest`
- `Ready`
- `FlightPlan`

The component starts in the initial pseudo-state and automatically reaches `SelfTest`. After receiving the setup message, the component moves to `Ready`. From `Ready`, it receives drone telecommands. If the execution of a telecommand makes a flight plan ready, the component enters `FlightPlan`. While in `FlightPlan`, the timer periodically triggers the control algorithm. When the flight plan is completed, the component returns to `Ready`.

5.2.2 Software Behaviour

TODO 01 – Scenarios

Scenario	Description
EvInit	Initial EDROOM start event. The component starts its behaviour and enters the SelfTest state.
EvDroneSelfTest	The component executes FDroneSelfTest() and calls pus_service129_drone_selftest() .
EvDroneSetUp	The component receives SDroneSetUp through DroneMngCtrl . The configuration data are handled by FGetConfig() .
EvDroneReady	After receiving the setup, the component executes FReplyDroneReady() and replies with SDroneReady .
EvDroneTC	The component receives SDroneTC through DroneMngCtrl . The telecommand is handled by FExecDroneTC() .
EvInitFlightPlan	After a TC execution, the branch InitFlightPlan is taken if GExecFlightPlan() returns true. The component executes FInitFlightPlan() and enters FlightPlan .
EvWaitFP	If no flight plan is ready, the default branch WaitFP is taken and the component remains in Ready .
EvCtrlAlgorithm	In FlightPlan , the component receives EDROOMSignalTimeout through DroneTimer from DroneTimer and executes FFlightCtrlAlgorithm() .
EvPlanDone	After the control algorithm, if GFlightPlanDone() returns true, the branch PlanDone is taken and FToReady() is executed.
EvProgNextCtrl	If the flight plan has not finished, the default branch ProgNextCtrl is taken and FProgNextCtrl() programs the next control activation.

Table 3: Scenarios involving **CCDroneMng**

TODO 02 – Sequence Diagrams

The EDROOM tool used in the project does not provide a specific sequence diagram editor for these scenarios. Therefore, the sequence behaviour is documented from the EDROOM state machine, the protocol definition and the generated behaviour code.

Scenario	Sequence
Drone setup sequence	DroneMngCtrl receives SDroneSetUp . The component executes FGetConfig() , applies the configuration through pus_service129_setup() , then executes FReplyDroneReady() and replies with SDroneReady .

Scenario	Sequence
Drone telecommand execution sequence	<code>DroneMngCtrl</code> receives <code>SDroneTC</code> . The component executes <code>FExecDroneTC()</code> . After that, the choice point evaluates <code>GExecFlightPlan()</code> . If true, <code>FInitFlightPlan()</code> is executed and the component enters <code>FlightPlan</code> ; otherwise, the component returns to <code>Ready</code> .
Periodic flight control sequence	While in <code>FlightPlan</code> , <code>DroneTimer</code> raises <code>EDROOMSignalTimeout</code> . The component executes <code>FFlightCtrlAlgorithm()</code> . Then the choice point evaluates <code>GFlightPlanDone()</code> . If true, <code>FToReady()</code> is executed and the component returns to <code>Ready</code> ; otherwise, <code>FProgNextCtrl()</code> programs the next control instant and the component remains in <code>FlightPlan</code> .

Table 4: Sequence behaviour of scenarios involving `CCDroneMng`

5.3 Real-Time Requirements

Requirement	Description
RTR-CCDM-001	While the component is in the <code>FlightPlan</code> state, the control algorithm shall be executed after receiving <code>EDROOMSignalTimeout</code> from <code>DroneTimer</code> .
RTR-CCDM-002	The next control activation shall be stored in <code>VNextCtrl</code> .
RTR-CCDM-003	The next activation shall be programmed with <code>DroneTimer.InformAt(time)</code> .
RTR-CCDM-004	The periodic control step used by the component is 100 ms, represented in the code as <code>Pr_Time(0,100000)</code> .

Table 5: Real-time requirements of `CCDroneMng`

5.4 Software Components Design

5.4.1 CCDroneMng

The `CCDroneMng` component manages the drone operational behaviour. It receives setup and telecommand requests, executes the corresponding service functions and controls the flight plan execution through a timer-driven state machine.

5.4.2 Interfaces

TODO 03 – Ports defining the interfaces of CCDroneMng

Port	Type	Description
DroneMngCtrl	External	Main control interface of CCDroneMng. It uses the protocol CPDroneMngCtrl.
DroneTimer	Timing	Timer interface used to receive timeout events for the periodic flight control algorithm.

Table 6: Ports of CCDroneMng

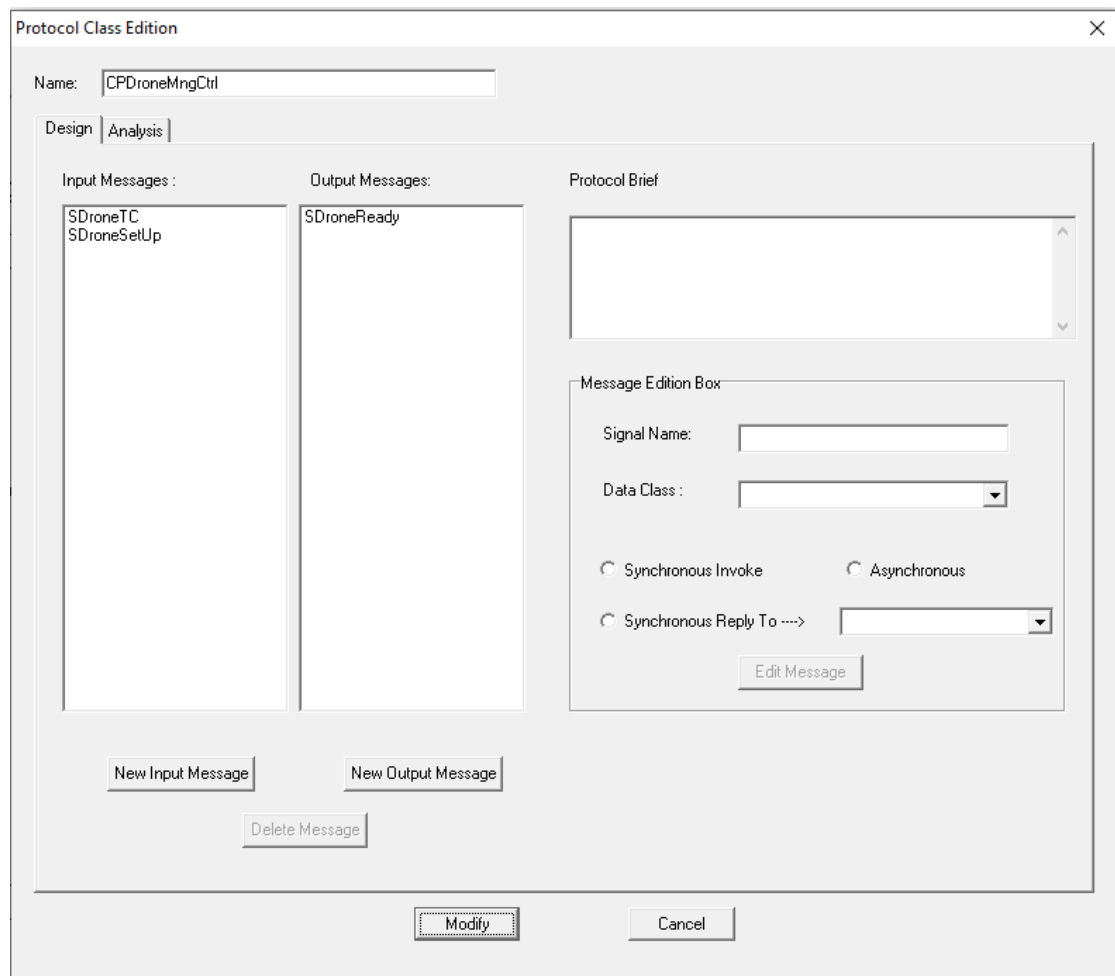


Figure 1: EDROOM protocol CPDroneMngCtrl used by CCDroneMng

Message	Direction	Data Class	Description
SDroneSetUp	Input	CDDroneConfig	Synchronous setup message used to configure the drone.
SDroneTC	Input	CDTCHandler	Message used to request the execution of a drone telecommand.
SDroneReady	Output	None	Synchronous reply indicating that the drone is ready.

Table 7: Messages of protocol CPDroneMngCtrl

5.4.3 Behaviour

TODO 04 – State diagram of CCDroneMng

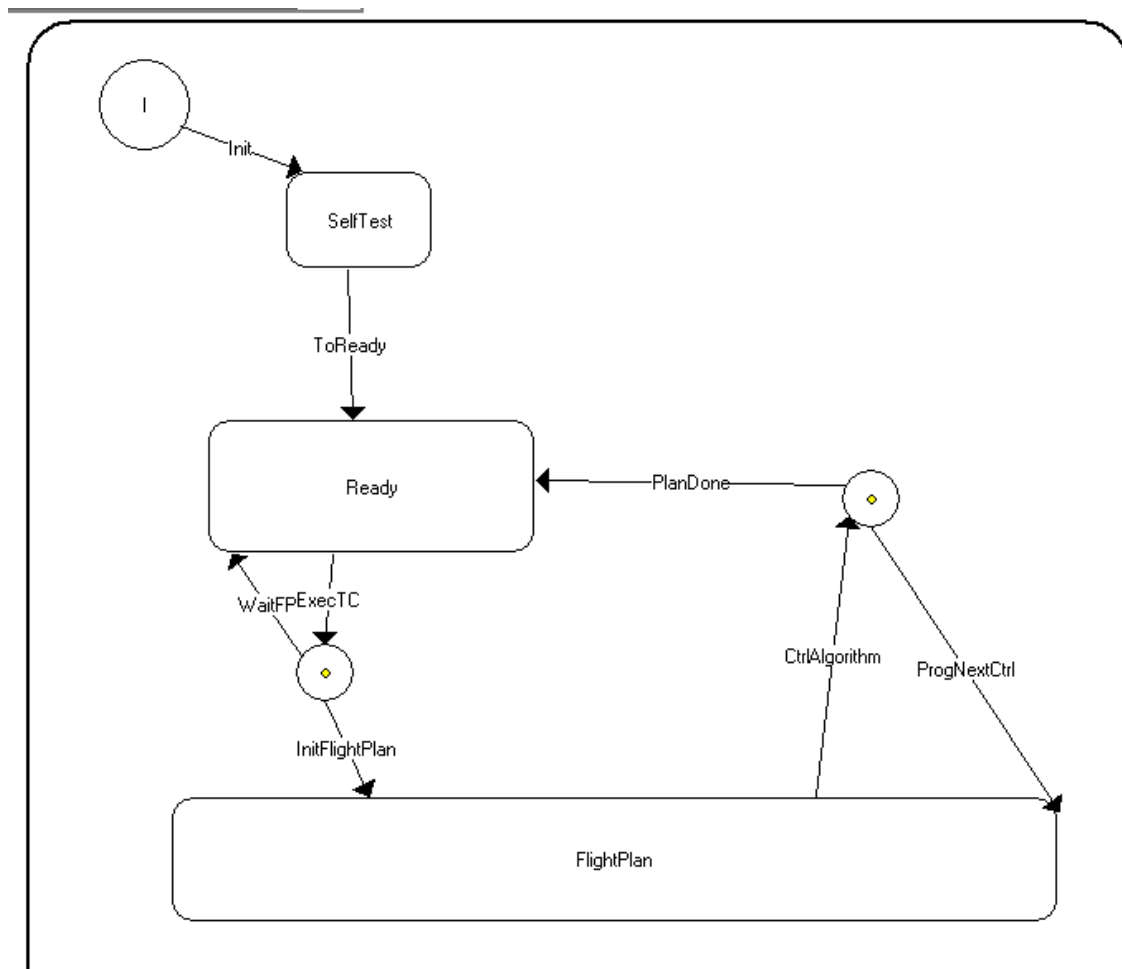


Figure 2: EDROOM statechart of CCDroneMng

The statechart contains the states **SelfTest**, **Ready** and **FlightPlan**. It also contains two choice points: one after telecommand execution and one after the periodic control algorithm.

TODO 05 – Variables of CCDroneMng

Variable	Type	Description
VNextCtrl	Pr_Time	Stores the next activation instant of the periodic flight control algorithm. It is updated when the flight plan starts and after each control cycle.

Table 8: Variables of CCDroneMng

TODO 06 – Trigger condition and action of each transition

Transition	Trigger	Guard	Action / Handler
Init	EDROOMStart through EDROOMs1	true	No transition action. Next state: SelfTest.
ToReady	SDroneSetUp through DroneMngCtrl	true	FGetConfig(); and FReplyDroneReady();. Next state: Ready.
ExecTC	SDroneTC through DroneMngCtrl	true	FExecDroneTC();. Then the choice point evaluates the flight plan condition.
CtrlAlgorithm	EDROOMSignalTimeout through DroneTimer	true	FFlightCtrlAlgorithm();. Then the choice point evaluates whether the flight plan has finished.

Table 9: Transitions of CCDroneMng

TODO 07 – Guards and actions of the choice point branches

Branch	Choice Point	Guard	Action
InitFlightPlan	After ExecTC	GExecFlightPlan()	FInitFlightPlan();
WaitFP	After ExecTC	true	No action.
PlanDone	After CtrlAlgorithm	GFlightPlanDone()	FToReady();
ProgNextCtrl	After CtrlAlgorithm	true	FProgNextCtrl();

Table 10: Choice point branches of CCDroneMng

TODO 08 – Code of actions and guards

The following code corresponds to the actions and guards associated with the EDROOM statechart of CCDroneMng.

Listing 1: Action FDroneSelfTest

```

1 void CCDroneMng::EDROOM_CTX_Top_0::FDroneSelfTest()
2 {
3     pus_service129_drone_selftest();
4 }

```

Listing 2: Action FExecDroneTC

```

1 void CCDroneMng::EDROOM_CTX_Top_0::FExecDroneTC()
2 {
3     CDTCHandler & varSDroneTC = *(CDTCHandler *)Msg->data;
4     varSDroneTC.ExecDroneTC();
5 }

```

Listing 3: Action FFlightCtrlAlgorithm

```

1 void CCDroneMng::EDROOM_CTX_Top_0::FFlightCtrlAlgorithm()
2 {
3     pus_service129_flight_ctrl_algorithm();
4 }

```

Listing 4: Action FGetConfig

```

1 void CCDroneMng::EDROOM_CTX_Top_0::FGetConfig()
2 {
3     CDDroneConfig & varSDroneSetUp = *(CDDroneConfig *)Msg->data;
4     pus_service129_setup(varSDroneSetUp);
5 }

```

Listing 5: Action FInitFlightPlan

```

1 void CCDroneMng::EDROOM_CTX_Top_0::FInitFlightPlan()
2 {
3     Pr_Time time;
4     time.GetTime();
5     time += Pr_Time(0,100000);
6     VNextCtrl=time;
7     pus_service129_init_flight_plan();
8     DroneTimer.InformAt(time);
9 }

```

Listing 6: Action FProgNextCtrl

```

1 void CCDroneMng::EDROOM_CTX_Top_0::FProgNextCtrl()
2 {
3     Pr_Time time;
4     VNextCtrl += Pr_Time(0,100000);
5     time=VNextCtrl;
6     DroneTimer.InformAt(time);
7 }

```

Listing 7: Action FReplyDroneReady

```

1 void CCDroneMng::EDROOM_CTX_Top_0::FReplyDroneReady()
2 {
3     pus_service129_drone_ready();
4     Msg->reply(SDroneReady);
5 }

```

Listing 8: Action FToReady

```

1 void CCDroneMng::EDROOM_CTX_Top_0::FToReady()
2 {
3     pus_service129_drone_ready();
4 }

```

Listing 9: Guard GExecFlightPlan

```

1 bool CCDroneMng::EDROOM_CTX_Top_0::GExecFlightPlan()
2 {
3     return pus_service129_flight_plan_ready();
4 }

```

Listing 10: Guard GFlightPlanDone

```

1 bool CCDroneMng::EDROOM_CTX_Top_0::GFlightPlanDone()
2 {
3     return pus_service129_flight_plan_done();
4 }

```

5.5 SRD Req Coverage Matrix

Requirement / Functionality	Covered by
Drone self-test	State SelfTest and action FDroneSelfTest().
Drone setup	Transition ToReady, message SDroneSetUp, action FGetConfig().
Drone ready reply	Action FReplyDroneReady() and message SDroneReady.
Drone telecommand execution	Transition ExecTC, message SDroneTC, action FExecDroneTC().
Flight plan start	Branch InitFlightPlan, guard GExecFlightPlan(), action FInitFlightPlan().
Periodic flight control	Transition CtrlAlgorithm, timer signal EDROOMSignalTimeout, action FFlightCtrlAlgorithm().
Next control cycle programming	Branch ProgNextCtrl and action FProgNextCtrl().
Flight plan completion	Branch PlanDone, guard GFlightPlanDone(), action FToReady().

Table 11: SRD requirement coverage matrix

6 Traceability

Architectural Element	Implementation Element
CCDroneMng component	DDF/uah_mars_drone_freertos_hito1/asw/components/ccdroneMng
DroneMngCtrl port	EDROOM port of CCDroneMng; protocol CPDroneMngCtrl.
DroneTimer port	EDROOM timing port used by the FlightPlan state.

Architectural Element	Implementation Element
SDroneSetUp	Input message of CPDroneMngCtrl; handled by transition ToReady.
SDroneTC	Input message of CPDroneMngCtrl; handled by transition ExecTC.
SDroneReady	Output message of CPDroneMngCtrl; sent by FReplyDroneReady().
VNextCtrl	Internal variable of CCDroneMng; type Pr_Time.
FGetConfig()	Implemented in CCDroneMngB.cpp; calls pus_service129_setup().
FReplyDroneReady()	Implemented in CCDroneMngB.cpp; calls pus_service129_drone_ready() and replies with SDroneReady.
FExecDroneTC()	Implemented in CCDroneMngB.cpp; executes the received CDTCHandler.
FInitFlightPlan()	Implemented in CCDroneMngB.cpp; starts the flight plan and programs DroneTimer.
FFlightCtrlAlgorithm()	Implemented in CCDroneMngB.cpp; calls pus_service129_flight_ctrl_algorithm().
FProgNextCtrl()	Implemented in CCDroneMngB.cpp; updates VNextCtrl and programs the next timer activation.
FToReady()	Implemented in CCDroneMngB.cpp; calls pus_service129_drone_ready().
GExecFlightPlan()	Implemented in CCDroneMngB.cpp; calls pus_service129_flight_plan_ready().
GFlightPlanDone()	Implemented in CCDroneMngB.cpp; calls pus_service129_flight_plan_done().

Table 12: Traceability matrix